

PetGuard AI

Computer Vision-Based Pet Restricted-Zone Monitoring

Tier 1 - Object Detection Application

PetGuard AI transforms ordinary camera footage into intelligent pet monitoring by detecting dogs and cats and determining when they enter user-defined restricted areas.



Team Members

1. Huong Nguyen
2. Sadia Saeed
3. Cherluk Sumdin
4. Mehmiyah Rouf

Class: ITAI-1378 Computer Vision

Professor: Patricia Mcmanus

Good
Pets
Safer
Homes
Happier
Humans
♥

They know
when we're
not watching...
♥

HAPPY
PETS
HAPPIER
PEOPLE
♥

Off Limits!
♥

Same Home.
Smarter Care. ♥



The Problem

"They Know When We're Not Watching." 

Happy
Pets
Better
Days 

Home cameras can record pets, but recording alone does not answer:

"Where is the pet, and is it somewhere it should not be?"



Real-world problem

Pet owners may want pets to stay away from:



Couch



Bed



Kitchen



Dining area



Other restricted areas

Reviewing camera footage manually is inefficient and provides little immediate context.



Computer-vision problem

PetGuard needs to solve two different questions:



Perception

What object is present, and where is it?



Spatial reasoning

Is that detected pet inside a restricted area?



Project objective

Develop a proof-of-concept system capable of:



Detecting a pet



Locating it



Evaluating its position



Generating a meaningful event



Key insight

Object detection tells us what and where.

PetGuard adds context to determine whether that location matters.



*Same Home.
Smarter Care.* 

System Architecture

From Pixels to Decisions 🐾



Any Camera.
Any Moment.
A Safer Home. ❤️



Live Camera



Video



Image



CAMERA / IMAGE / VIDEO

Capture pet activity from home cameras or uploaded media



FRAME INPUT

Extract frames from video or use image



PRETRAINED YOLO

OBJECT DETECTOR
Detect objects in the frame



DOG / CAT FILTER

Keep only dog and cat detections



CONFIDENCE THRESHOLD

Filter low-confidence detections



PET BOUNDING BOX

Get location (x, y, width, height)



SPATIAL ZONE REASONING

"Where is the pet?"
Check if the pet is inside a restricted area



TEMPORAL CONFIRMATION

"Is it persistent?"
Validate across multiple frames



SAFE

Pet is outside
restricted areas



VIOLATION

Pet is inside
a restricted area

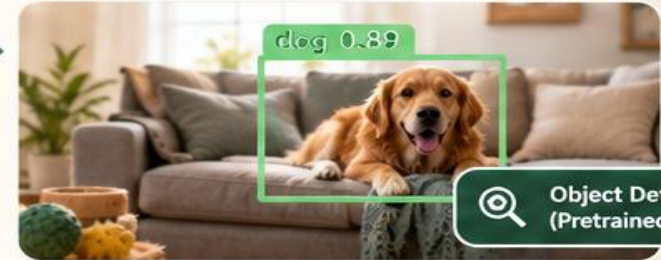


EVENT LOG / ALERT

Save event, notify user, and provide visual evidence



Same Pets.
Smarter Insights. ❤️



Object Detection
(Pretrained YOLO)



Spatial Zone
Reasoning



Temporal
Confirmation



Real Moments.
Real Alerts.
Peace of Mind. ❤️

Good Pets
Brighter Days. 🐾

Technical Approach

Detection + Spatial Intelligence 🐾

From
Images to Insights
for Happier Pets ❤️

dog 0.88



Good
Dogs
Brighter
Days ❤️

SAME HOME
SMARTER CARE 🐾



Computer Vision Technique

Object Detection



Baseline Model

YOLOv8n
with pretrained
COCO weights



Technology

Python • Ultralytics
PyTorch • OpenCV
Google Colab

Why YOLO?

YOLO is appropriate because PetGuard requires both:



Classification
Dog or cat?

+



Localization
Where is the pet?

For input frame I_t :

$$I_t \rightarrow \text{YOLO} \rightarrow \{B_i, C_i, S_i\}$$

where:

- B_i = bounding box
- C_i = predicted class
- S_i = confidence score

PetGuard retains detections where:

$$C_i \in \{\text{dog, cat}\}$$



Dogs and Cats
Welcome
(But in the Right Places!) ❤️



Current Technical Proof



Test Image



YOLOv8n



1 Dog Detected

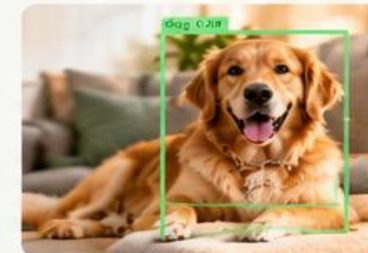


Confidence $\approx$ 87.9%



Bounding Box Extracted

Example Result (from Colab)



1 object detected:
- class: dog
- confidence: 0.879
- box: [102, 56, 628, 478]

Successfully detected the dog with $\sim$ 87.9% confidence! 🐾



The project is not purely theoretical.
We already validated the first technical dependency.

Small
Models
Big Impact! ❤️

Real
Scenarios
Real Impact 🐾

Data Strategy

Data With a Purpose 🐾

Not just
cute photos.
Real situations.
Real insights. 🐾

A Detection Baseline

Use pretrained YOLO weights (YOLOv8n, COCO).



YOLOv8n
(pretrained COCO)

Dog and cat are existing object classes, allowing us to establish an end-to-end baseline before considering model adaptation.



Initial Target

Start with approximately **100 – 200** evaluation images/frames. Then expand if necessary. Use a mix of public images and controlled test images/video.



Variety
builds a stronger,
smarter model. 🐾

B PetGuard Evaluation Set

Build a targeted evaluation set with real-world scenarios.



Pet outside zone
(SAFE)



Pet inside zone
(VIOLATION)



Pet at boundary



Partial occlusion



Different lighting



Different viewing
angles



Multiple pets



No pet present



Data Record

Maintain ground truth such as:

image_id	pet_present	pet_type	expected zone_status	predicted zone_status	confidence	notes
001	Yes	Dog	Violation	Violation	0.91	Clear view
002	Yes	Cat	Safe	Safe	0.86	Side angle
003	Yes	Dog	Safe	Violation	0.73	Boundary case
...	...	...	...	...	...	...



Data Principle

We will evaluate the system on situations that challenge the application — not just easy pet photographs.



Real Data.
Real Homes.
Real Protection. 🐾

Diverse Data
Safer Pets
Happier Homes 🐾

How PetGuard Reasons

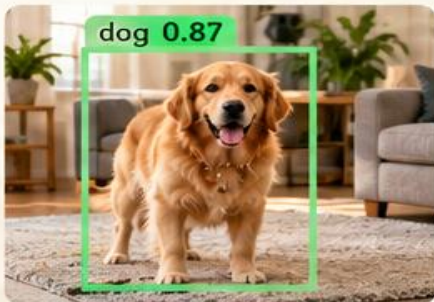
Detection $\neq$ Understanding 🐾

It's not just
what they are,
but where they are.

Same pet.
Different story.
Different logic.
Smarter decisions.

1 YOLO Detection

YOLO detects pets and returns a bounding box.

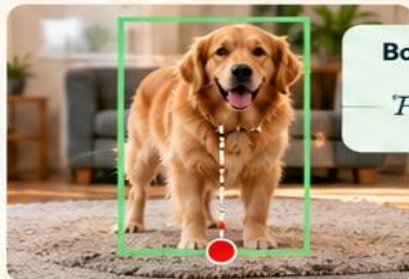


$$B = (x_1, y_1, x_2, y_2)$$

B = bounding box
 C = predicted class
 S = confidence score

2 Bottom-Center Point

Instead of the center of the box, we use the bottom-center point to represent the pet's location on the ground.



Bottom-center point

$$P = \left(\frac{x_1 + x_2}{2}, y_2 \right)$$

Closer to where
the pet touches
the ground! 🐾

3 Restricted Zone (Polygon)

Define a restricted zone $Z = \{p_1, p_2, \dots, p_n\}$ as a polygon. PetGuard checks if the pet's point is inside the zone.



$$\text{Violation} = \begin{cases} 1 & P \in Z \\ 0 & P \notin Z \end{cases}$$

Restricted Zone (Z) 🐾

4 Furniture Scenario

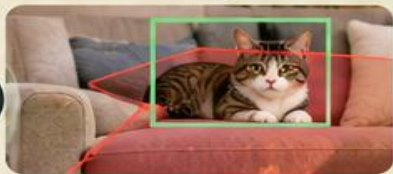
For furniture (e.g., couch or bed), bottom-center may not be ideal. We compare two methods:

Method A: Point-in-Polygon



Check if the bottom-center point is inside the zone.

Method B: Bounding-Box Overlap



Check the overlap between the bounding box and the zone.

VS



Different scenarios need different reasoning.
We'll evaluate both methods to find what works best. 🌿

5 Temporal Reasoning

A single frame should not necessarily create an alert.
We check for persistent violations across multiple frames.

Not Confirmed



Do not confirm
(brief crossing or false detection)

Confirmed Violation



CONFIRMED VIOLATION
(persistent behavior)



Why Temporal Reasoning?

- ✓ Reduces false alerts from bounding-box jitter
- ✓ Handles momentary detection errors
- ✓ Avoids alerts from brief zone crossing
- ✓ More reliable and practical
- ✓ Makes PetGuard a decision system

Smarter
AI for
happier
homes. ❤️



Good
Pets
Brighter
Days. ❤️

From Detections to Decisions

Understand the scene. Respect the space. Keep pets safe. 🐾

Success Metrics

"Working" Must Be Measurable 🐾

A successful system is more than detecting pets — it's about making the right decisions, with low false alerts, and practical performance.

Measure what matters.
Build what works.
Happier homes. 🌱

GOOD
PETS
BRIGHTER
DAYS
♥

1 Pet Detection Recall

Of the pets actually present, how many did YOLO detect?

$$\text{Recall} = \frac{TP}{TP + FN}$$



Initial target
≥ 85%



Detected
(True Positive)



Missed
(False Negative)

2 Zone Decision Accuracy

How often did PetGuard correctly classify SAFE vs VIOLATION?

$$\text{Accuracy} = \frac{\text{Correct Zone Decisions}}{\text{Total Evaluated Cases}}$$



Initial target
≥ 90%



Correct decision



Correct decision

3 False Violation Rate

How often does PetGuard incorrectly alert when the pet is actually safe?



Target
≤ 10%



Should be SAFE,
but system incorrectly alerted
(False Positive)

4 Processing Performance

Measure inference time or FPS
(milliseconds/frame or frames per second).



Initial focus
Reliable inference
(over chasing real-time)



Current experiment (one run):

~ 196 ms per frame

Performance may vary by hardware and environment. We will optimize after the baseline system is working reliably.

Same Home.
Happier Tails. ♥

Success = More Than Accuracy



Detection
(pet recall)

+



Correct
Spatial Decision
(SAFE vs VIOLATION)

+



Low
False Alerts

+



Practical
Processing Time

=

A Reliable,
Real-World
PetGuard AI

Right
detections.
Fewer false alerts.
Happier pets. ♥

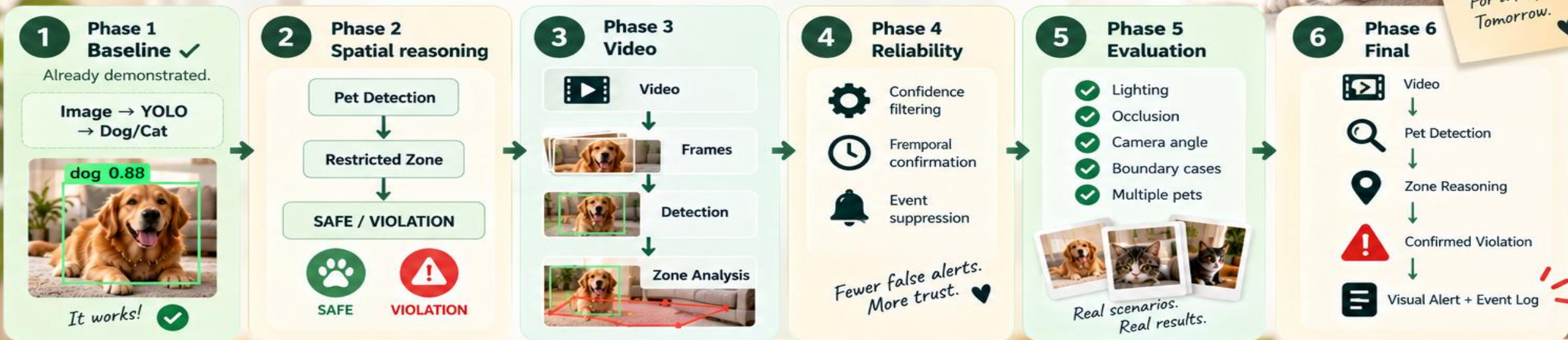
Pets make.
a house a home. 🐾

Execution Plan + Risks

Build Small → Validate → Expand

Small Steps
Big Impact
Happier Homes

Plan.
Build.
Test.
Improve.
For a Happier
Tomorrow.



Risk Assessment

Anticipate challenges. Plan ahead. Keep the project on track.

Risk	Impact	Mitigation / Plan B
Pet not detected	High	Confidence tuning; controlled evaluation
Occlusion	Medium	Include occlusion test cases
Poor lighting	Medium	Evaluate separately; document limitation
Zone-boundary ambiguity	High	Compare point vs overlap methods
Bounding-box jitter	Medium	Temporal confirmation
Video too slow	Medium	Reduce resolution/frame sampling
Repeated alerts	Medium	Event cooldown/debouncing
Multiple pets	Medium	Evaluate each detected pet separately
Complex custom zones	Low	Use predefined polygon
Dataset limitations	Medium	Controlled evaluation set
Custom training becomes necessary	High	Remain with pretrained baseline for Tier 1
Final scope becomes too large	High	Protect minimum viable pipeline



Most Important Plan B

If things become difficult, we still have a clear, achievable final.



A solid plan
ensures a
successful final!

Better AI
for Brighter Tomorrows

